【扔物线 Compose 课程讲义】7.3.5-自定义触摸：最底层的100%自定义触摸算法

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当节课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：https://rengwuxian.com/cp

基本写法

- Modifier.pointerInput()
- awaitEachGesture()
- 里面用 `awaitPointerEvent()` 获取和处理事件，定制自己的算法。

关于触摸事件

- 按下和抬起事件跟 View 系统的不同： `Press` 和 `Release` 包含了任何触摸点的按下和抬起，

所以对于「第一次按下」和「最后一次抬起」，要做额外的附加条件来判断（例如「只有一个触摸点存在」）。
- 触摸事件是「有触摸点发生了事件」，而不是「某个触摸点发生了事件」，因此 `PointerEvent` 的 `changes` 是个数组。

消费、拦截和取消

- Compose 里是三遍事件传递的流程，而不是 Android 原生的两遍。
- Compose 不再有「事件拦截」的概念，而是在任何节点都可以消费，并可以被后续的节点感知到「已消费」的事实。
- 原则：
  - 使用触摸事件之后，把它消费掉（如果有合适的理由，可以例外）；
  - 使用触摸事件之前，先看它有没有消费，没有消费过再触发业务逻辑（如果有合适的理由，可以例外）；
  - 另外：记得考虑一下要不要在第三流程里检查事件是否被父组件消费过，从而取消手势。